

Федеральное государственное бюджетное образовательное  
учреждение высшего образования

РЫБИНСКИЙ ГОСУДАРСТВЕННЫЙ АВИАЦИОННЫЙ  
ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ П.А. СОЛОВЬЕВА

Факультет радиоэлектроники и информатики

Кафедра математического и программного обеспечения электронных  
вычислительных средств

## **КУРСОВОЙ ПРОЕКТ**

по дисциплине

«Базы данных»

на тему:

«Торговая фирма»

## **ПОЯСНИТЕЛЬНАЯ ЗАПИСКА**

Руководитель

Шаров В.Г., Задорина Н.А.

Рыбинск 2023

## Оглавление

|                                     |    |
|-------------------------------------|----|
| 1. Введение.....                    | 3  |
| 2. Постановка задачи.....           | 5  |
| 3. Описание предметной области..... | 6  |
| 4. Инфологическая модель.....       | 8  |
| 5. Даталогическая модель.....       | 10 |
| 6. Реализация базы данных.....      | 12 |
| 7. Реализация приложения.....       | 16 |
| 8. Заключение.....                  | 25 |
| 9. Список литературы.....           | 26 |
| 10. Приложения.....                 | 27 |

# **1. Введение**

## **Актуальность разработки**

Актуальность разработки программы «Торговая фирма» заключается в автоматизации ведения учета заказанных товаров, товаров на складе, поставщиков товара, а также иметь инфографику по товарам и создавать отчеты по имеющимся товарам.

## **Цель работы и постановка задачи**

Цель работы создать кросс-платформенное приложение, которое могло бы упростить процесс взаимодействия фирмы с клиентами посредством удобного интерфейса и удобного хранения данных.

Торговая фирма занимается оптовой торговлей. Фирма имеет сеть поставщиков товаров. Поставщики находятся в разных городах. Каждый поставщик специализируется на поставке одного товара, склады фирм позволяют разместить ограниченное количество каждого из товаров.

## **Технические и программные средства реализации**

Система «Торговая фирма» представляет собой приложение со «встраиваемым» движком PostgreSQL.

Основная часть системы реализована с помощью кроссплатформенного фреймворка для разработки программного обеспечения на языке программирования Java – Swing. С его помощью реализована интерфейсная и логическая часть программы, обеспечивающая визуальное представление для работы с программой, и содержащую в себе основные инструменты для работы с базой данных. Клиент вызывает определенные функции API (Application Programming Interface) с параметрами в виде запросов (в нашем случае запросы, сформированные на языке SQL), движок PostgreSQL обрабатывает и выполняет запросы и возвращает необходимые данные клиенту.

Благодаря PostgreSQL, не требуется использовать технологию «клиент-сервер», так как движок PostgreSQL не является отдельно работающим процессом, с которым взаимодействует программа, а представляет собой библиотеку, с которой программа компонуется, и движок становится составной частью программы. Такой подход уменьшает накладные расходы, время отклика и упрощает программу. PostgreSQL

хранит всю базу данных (включая определения, таблицы, индексы и данные) в единственном стандартном файле на том компьютере, на котором выполняется программа. Несколько процессов или потоков могут одновременно без каких-либо проблем читать данные из одной базы. Запись в базу можно осуществить только в том случае, если никаких других запросов в данный момент не обслуживается.

## 2. Постановка задачи

### Торговая фирма

Торговая фирма занимается оптовой торговлей. Фирма имеет сеть поставщиков товаров. Поставщики находятся в разных городах. Каждый поставщик специализируется на поставке одного товара, склады фирм позволяют разместить ограниченное количество каждого из товаров.

Товар на фирму может поступать по ее заявкам или по предложению поставщика без предварительного заказа. Отпуск товаров производится по заявкам.

Система должна обеспечивать выполнение следующих операций:

- прием предложения поставщика о возможности поставить товар в виде накладной;
- прием товара на склад или отказ в приеме;
- формирование заказа поставщику на поставку товара;
- отпуск товара или отказ;
- получение справок о сделанных заказах поставщикам, предложениях поставщиков, о заявках на отпуск товаров;
- получение других справок.

### 3. Описание предметной области

#### Общее описание предметной области

При процессе подачи заявки на товар клиент обращается в торговую фирму, где заполняет специальную форму, указывая необходимые товары и их количество. После заполнения формы клиент передает ее сотруднику фирмы, который затем принимает заявку, проверяя её корректность и наличие необходимых товаров на складе. Если данные в заявке корректны и товары имеются, сотрудник принимает заявку для дальнейшей обработки.

Предметная область подразумевает выполнение следующих функций:

- Добавление нового товара;
- Удаление товара из списка;
- Добавление поставщика;
- Удаление поставщик;
- Добавление заказа;
- Удаление заказа
- Добавление склада;
- Удаление склада;
- Просмотр подробностей о всех данных из столбцов
- Отмену или принятие заказа от клиента;
- Просмотр подробностей о товаре.
- Просмотр подробностей о складе.
- Просмотр подробностей о поставщике.
- Просмотр подробностей о заказе.
- Отмену или принятие заявки от клиента.
- Диаграмма количества имеющихся товаров в процентном соотношении.
- Отчет, формируемый о товаре в формате .csv

## **Описание входных документов**

Входными являются данные, которые заполняются в специальных полях программы. Для клиента – оформление заявки на отпуск товара, а для администратора это добавление/удаление товара и склада и принять/отклонить заявку от клиента. При подтверждении ввода данных происходит обработка введенной информации и заполняются поля базы данных, либо выводится сообщение о некорректных данных.

## **Описание выходных документов**

Выходными документами являются справки о товаре (приложение 1). Они формируются на основе данных: название товара, название поставщика, данные о местоположении поставщика, данные об адресе доставки, цене товара (штучно), цене товара (общая цена за все товара), количестве заказанного товара и данных пользователя.

## **Список ограничений**

- Размер строковых полей должен быть не более 255 символов
- Вещественные и целые числа обязательно должны быть положительными.
- Диапазон значений целых чисел от 0 до 2147483647.
- Размер числовых полей не должен превышать 20 знаков.

## 4. Инфологическая модель

В ходе изучения одной из заявок торговой фирмы мы пришли к выводу, что самым оптимальным выбором для создания базы данных будет тот, что описан далее. Он будет самым оптимальным и универсальным.

### Сущности и их ключевые атрибуты:

- Товар (кодТовара)
- Склад (кодСклад)
- Заявка (кодЗаявка)
- Поставщик товара (кодПоставщик)
- Логин (Логин)

### Бизнес правила

#### Заявка-Товар

- Заявка может включать в себя несколько товаров.
- Товар может быть включен в несколько заявок.
- Товар может быть не включен ни в одну заявку.
- Заявка должна включать хотя бы один товар.

#### Склад-Товар

- Товар может быть размещен на нескольких складах.
- На складе может быть размещено несколько товаров.
- Товар не обязательно размещен на складе.
- На складе должен быть размещен хотя бы один товар.

#### Товар-Поставщик товара

- Товар может быть поставлен несколькими поставщиками.
- Поставщик может поставлять только один товар.
- Товар должен поставляться каким-либо поставщиком.
- Поставщик должен поставлять какой-либо товар.

#### Логин-Клиент

- Клиент может иметь только один логин.
- Логин может принадлежать только одному клиенту.
- У клиента обязательно должен быть логин.
- Каждому логину должен соответствовать клиент.



## ER–диаграмма

ER–диаграмма приведена на рисунке 1.

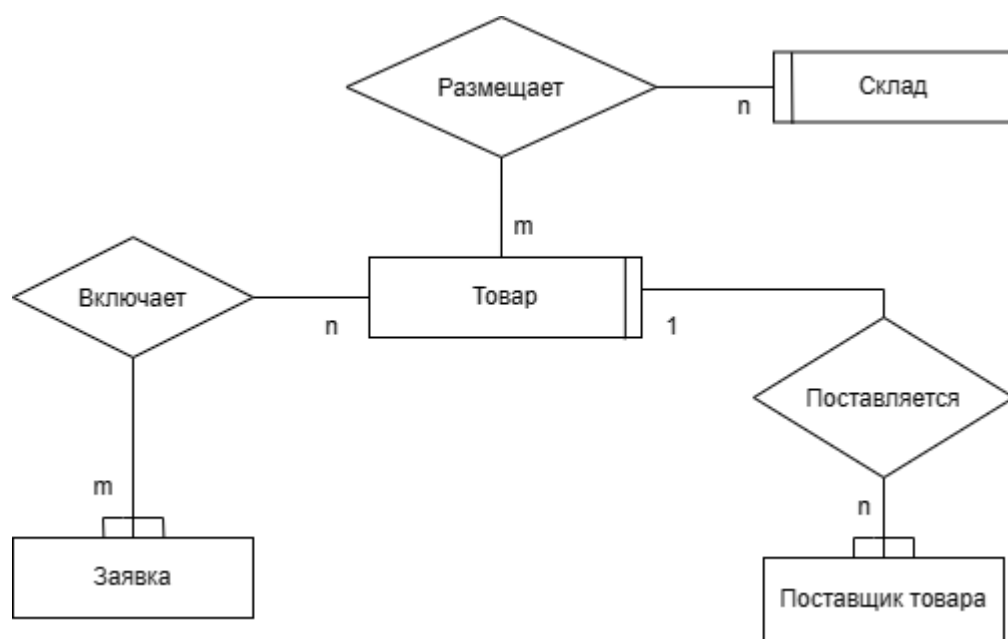


Рисунок 1 – ER–диаграмма

## 5. Даталогическая модель

### Отношения базы данных

- product(id, ...)
- supplier(id, ...)
- requisition(id, ...)
- storage(id, ...)
- requisition\_product(requisition\_id, product\_id, ...)
- storage\_product(storage\_id, product\_id, ..)
- user(id, ...)

### Описание структуры всех отношений БД

| Наименование информационного объекта | Наименование атрибута | Имя атрибута в ИЛМ  | Тип  | Длина | Точность | Признак ключа |
|--------------------------------------|-----------------------|---------------------|------|-------|----------|---------------|
| Product                              | КодПродукта           | id                  | I    | 20    | -        | *             |
|                                      | Наименование          | name                | S    | 255   | -        |               |
|                                      | Цена                  | price               | D    | 10    | 2        |               |
| Supplier                             | КодПоставщика         | id                  | I    | 20    | -        | *             |
|                                      | Наименование          | name                | S    | 255   | -        |               |
|                                      | Местоположение        | location            | S    | 255   | -        |               |
|                                      | Контактная информация | contact_information | S    | 255   | -        |               |
|                                      | КодПродукта           | product_id          | I    | 20    | -        | *             |
| Requisition                          | КодЗаявки             | id                  | I    | 20    | -        | *             |
|                                      | Статус                | status              | S    | 255   | -        |               |
|                                      | Дата                  | date                | Date | -     | -        |               |
|                                      | Код пользователя      | user_id             | I    | 20    | -        | *             |
| Storage                              | КодСклада             | id                  | I    | 20    | -        | *             |
|                                      | Местоположение        | location            | S    | 255   | -        |               |
|                                      | Вместимость           | capacity            | I    | 20    | -        |               |
|                                      | Заполненность         | occupancy           | I    | 20    | -        |               |
| Requisition_Product                  | КодЗаявки             | requisition_id      | I    | 20    | -        | *             |
|                                      | КодПродукта           | product_id          | I    | 20    | -        | *             |

| Наименование информационного объекта | Наименование атрибута | Имя атрибута в ИЛМ | Тип | Длина | Точность | Признак ключа |
|--------------------------------------|-----------------------|--------------------|-----|-------|----------|---------------|
|                                      | Количество            | quantity           | I   | 20    | -        |               |
|                                      | Цена                  | price              | D   | 10    | 2        |               |
| Storage_Product                      | КодСклада             | storage_id         | I   | 20    | -        | *             |
|                                      | КодПродукта           | product_id         | I   | 20    | -        | *             |
|                                      | Количество            | quantity           | I   | 20    | -        |               |
| User                                 | КодПользователя       | id                 | I   | 20    | -        | *             |
|                                      | Логин                 | login              | S   | 255   | -        |               |
|                                      | Пароль                | password           | S   | 255   | -        |               |
|                                      | Роль                  | role               | S   | 255   | -        |               |

### Сводная таблица отношений

| Имя таблицы         | Описание              |
|---------------------|-----------------------|
| Product             | Таблица продуктов     |
| Supplier            | Таблица поставщиков   |
| Requisition         | Таблица заявок        |
| Storage             | Таблица складов       |
| Requisition_Product | Таблица включает      |
| Storage_Product     | Таблица хранения      |
| User                | Таблица пользователей |

Так как все атрибуты в таблицах являются атомарными (неделимыми), то таблица уже находится в первой начальной форме (1NF). Далее, все неключевые атрибуты в таблицах полностью зависят от первичного ключа. В таблицах нет частичной зависимости атрибутов от составного первичного ключа-значит во второй нормальной форме (2NF). И наконец, в бд нет транзитивных зависимостей (3NF).

## 6. Реализация базы данных.

Работа с базой данных осуществляется с помощью встроенного движка PostgreSQL. В качестве протокола обмена используются вызовы функций (API) библиотеки SQLite. Данные обработчики всегда вызывают функции, которые формируют SQL-запросы к СУБД и возвращают ответы в виде списка объектов конкретной структуры данных.

СУБД не использует ни триггеры, ни хранимые процедуры. Все запросы формируются динамически в приложении.

### Создание таблиц в БД

```
CREATE TABLE "storage" (  
    id bigint NOT NULL,  
    name character varying(255) NOT NULL,  
    price double precision,  
    quantity bigint  
  
);  
  
CREATE TABLE "product" (  
    id bigint NOT NULL,  
    name character varying(255) NOT NULL,  
    price double precision,  
  
);  
  
CREATE TABLE "user" (  
    id integer NOT NULL,  
    login character varying(255),  
    password character varying(255),  
    role character varying(255)  
  
);  
  
CREATE TABLE "requisition" (  
    id bigint NOT NULL,  
    status character varying(255) NOT NULL,  
    date timestamp(6) without time zone NOT NULL,  
    user_id integer  
  
);  
  
CREATE TABLE "requisition_product" (  
    requisition_id bigint NOT NULL,  
    product_id bigint NOT NULL,  
    quantity bigint NOT NULL,  
    price double precision NOT NULL  
  
);
```

```

CREATE TABLE "storage_product" (
    storage_id bigint NOT NULL,
    product_id bigint NOT NULL,
    quantity bigint DEFAULT 0 NOT NULL
);
CREATE TABLE "supplier" (
    id bigint NOT NULL,
    name character varying NOT NULL,
    location character varying NOT NULL,
    contact_information character varying NOT NULL,
    product_id bigint NOT NULL
);

```

### **Примеры некоторых динамически формируемых SQL-запросов**

- Получение списка товаров с их количеством и ценой.

```

SELECT p, SUM(sp.quantity), p.price
FROM StorageProduct sp
JOIN Product p ON sp.productId = p.id
GROUP BY p.id, p.name, p.price

```

- Получение списка товаров в заявке.

```

FROM RequisitionProduct rp WHERE
rp.requisitionId = :requisitionId

```

- Возврат количества товаров на склад.

```

FROM StorageProduct sp WHERE sp.productId =
:productId

```

- Удаление товара.

```

DELETE FROM public.product WHERE name = ?

```

- Удаление заявки.

```

DELETE FROM public.requisition WHERE name = ?

```

- Удаление поставщика.

```
DELETE FROM public.supplier WHERE name = ?
```

- Удаление склада.

```
DELETE FROM public.storage WHERE name = ?
```

- Найти все склады по их локации.

```
select * from public.storage where location = ?
```

- Найти локацию и контактную информацию поставщика из заявки по имени поставщика.

```
select location, contact_information from  
public.requisition where name =
```

- Найти все товара по имени.

```
select * from public.product where name = ?
```

- Найти все заявки для продукта по id заявки.

```
select * from public.requisition_product where  
requisition_id = ?
```

- Найти id пользователя и статус из заявки по id заявки.

```
select user_id, status from public.requisition  
where id = ?
```

- Найти имя товара по его id.

```
select name from public.product where id = ?
```

- Получение списка названий, цен и описаний всех блюд, на которые можно оформить предварительный заказ.

```
SELECT Name, Price, Opisanie FROM Blyudo
```

- Обновить статус заявки на “ЗАВЕРШЕНА” по id заявки.

```
UPDATE public.requisition SET status =  
RequisitionStatus.COMPLETED WHERE id = ?
```

- Обновить статус заявки на “ОТМЕНЕНА” по id заявки.

```
UPDATE public.requisition SET status =  
RequisitionStatus.CANCELED WHERE id = ?
```

- Добавить склад.

```
INSERT INTO public.storage (name, price ,  
quantity) VALUES (?, ?, ?)
```

- Добавить заявку.

```
INSERT INTO public.requisition (location,  
capacity, occupancy) VALUES (?, ?, ?)
```

- Добавить поставщика.

```
INSERT INTO public.supplier (name, location,  
contact_information, product_id) VALUES (?, ?, ?, ?)
```

- Добавить товар.

```
INSERT INTO public.product (name, price ,  
quantity) VALUES (?, ?, ?)
```

## 7. Реализация приложения

Схема взаимодействия технических средств представлена на рисунке 2.

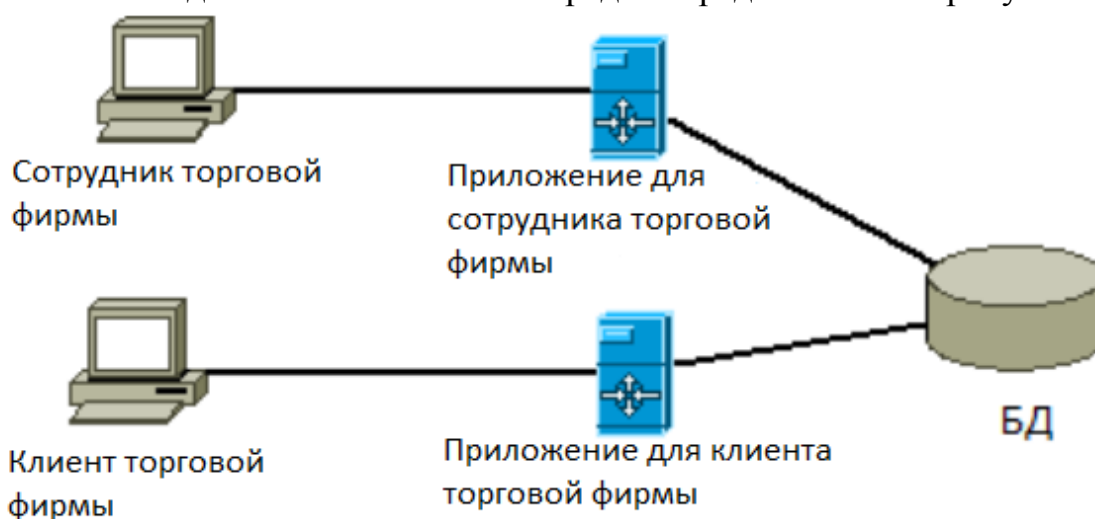


Рисунок 2 – Схема взаимодействия технических средств.

### Общая схема работы программы.

При запуске программы, перед пользователем появится окно авторизации (Рисунок 3). В зависимости от логина и пароля, системой будет определено, кем является пользователь – клиентом или работником фирмы. Все логины и пароли хранятся в базе данных. При вводе неправильной комбинации логина и пароля будет выведено соответствующее сообщение – «Неверный логин или пароль»

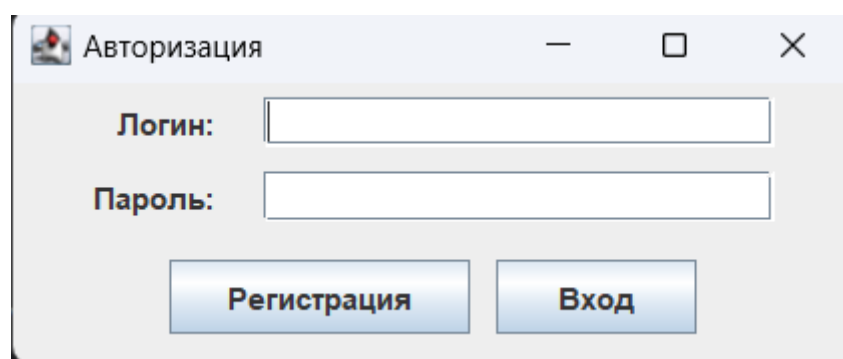


Рисунок 4–Авторизация пользователя.

У пользователя есть возможность зарегистрироваться. При нажатии на кнопку «Регистрация» отобразится соответствующее окно (Рисунок 5). Клиент может ввести желаемый логин, пароль и роль который будет в дальнейшем использовать для авторизации. Все поля является



обязательным для заполнения. При успешной регистрации клиент увидит сообщение «Регистрация прошла успешно». Введенные логин, пароль будут записаны в базу данных и потом их можно будет использовать для авторизации, роль также будет записана в базу данных и по ней программа будет определять какое приложение запустить для пользователя.

При ошибке регистрации будет выведено сообщение – «Все поля должны быть заполнены» или «Логин уже существует».

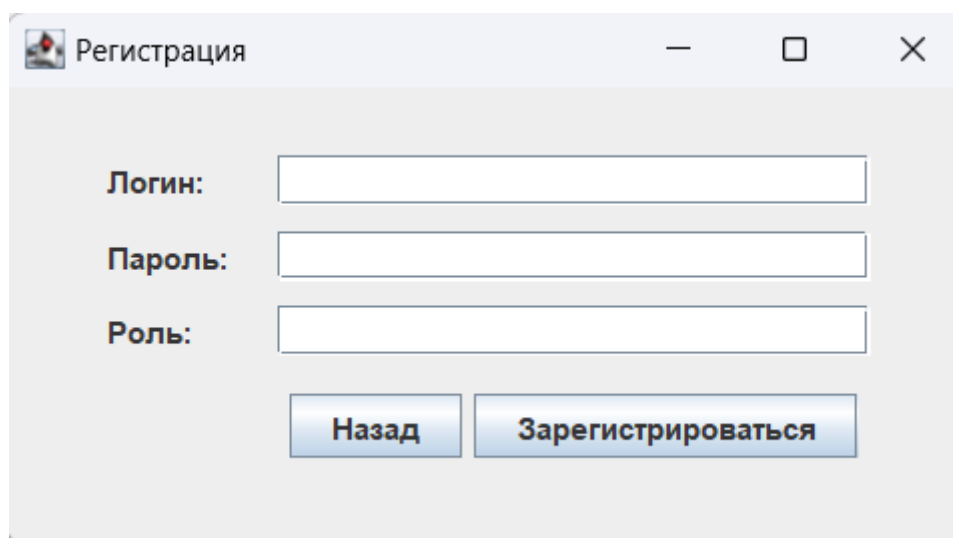


Рисунок 5 – Регистрация нового пользователя.

Если клиент успешно авторизовался, то перед ним откроется окно оформления заказа (Рисунок 6).

В окне показывается имя пользователя, доступные для заказа товары и корзина с заказами.

При двойном нажатии по товару, программа предложит ввести количество товара, которое вы хотите заказать (Рисунок 7). И товар, его количество и цена переместятся в корзину, а поле «Итого» заполнится итоговой ценой за все товары в корзине, с итогом количества (Рисунок 8).

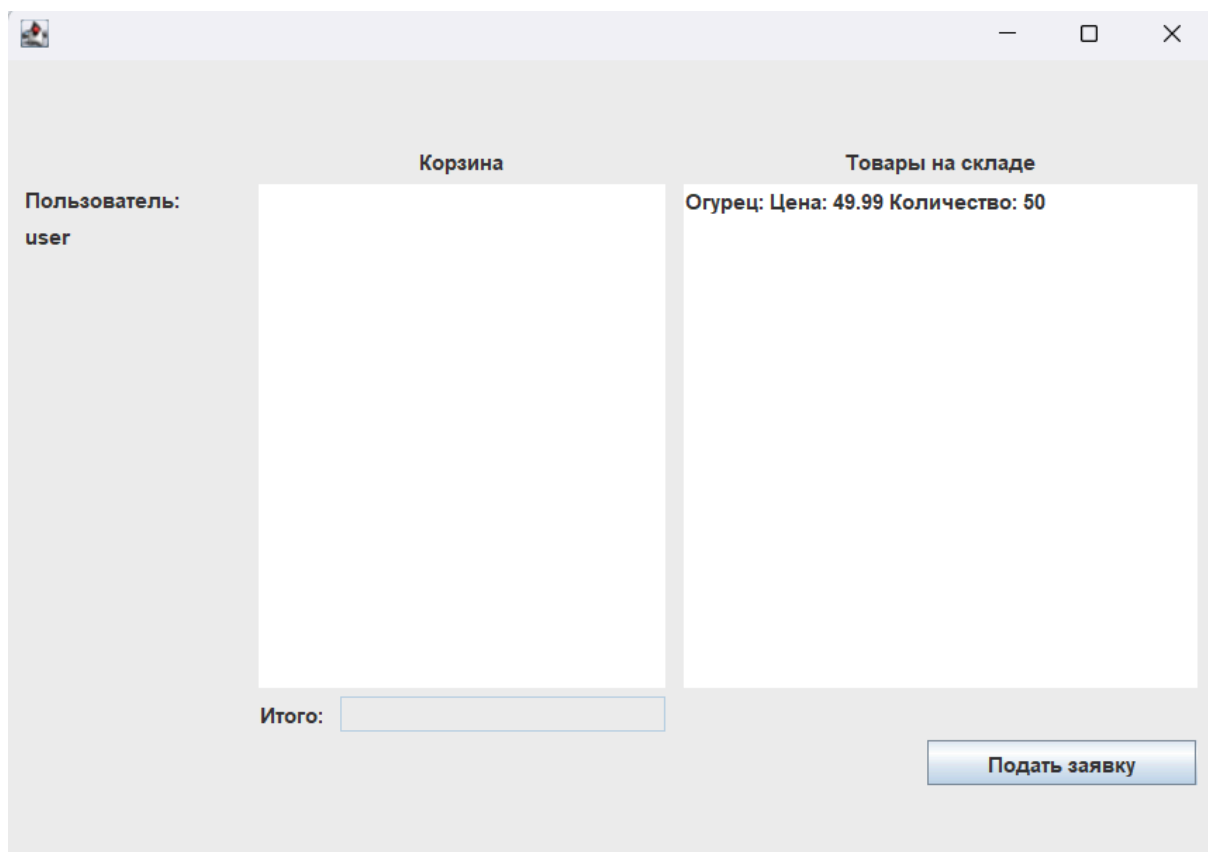


Рисунок 6 – Клиентское окно программы.

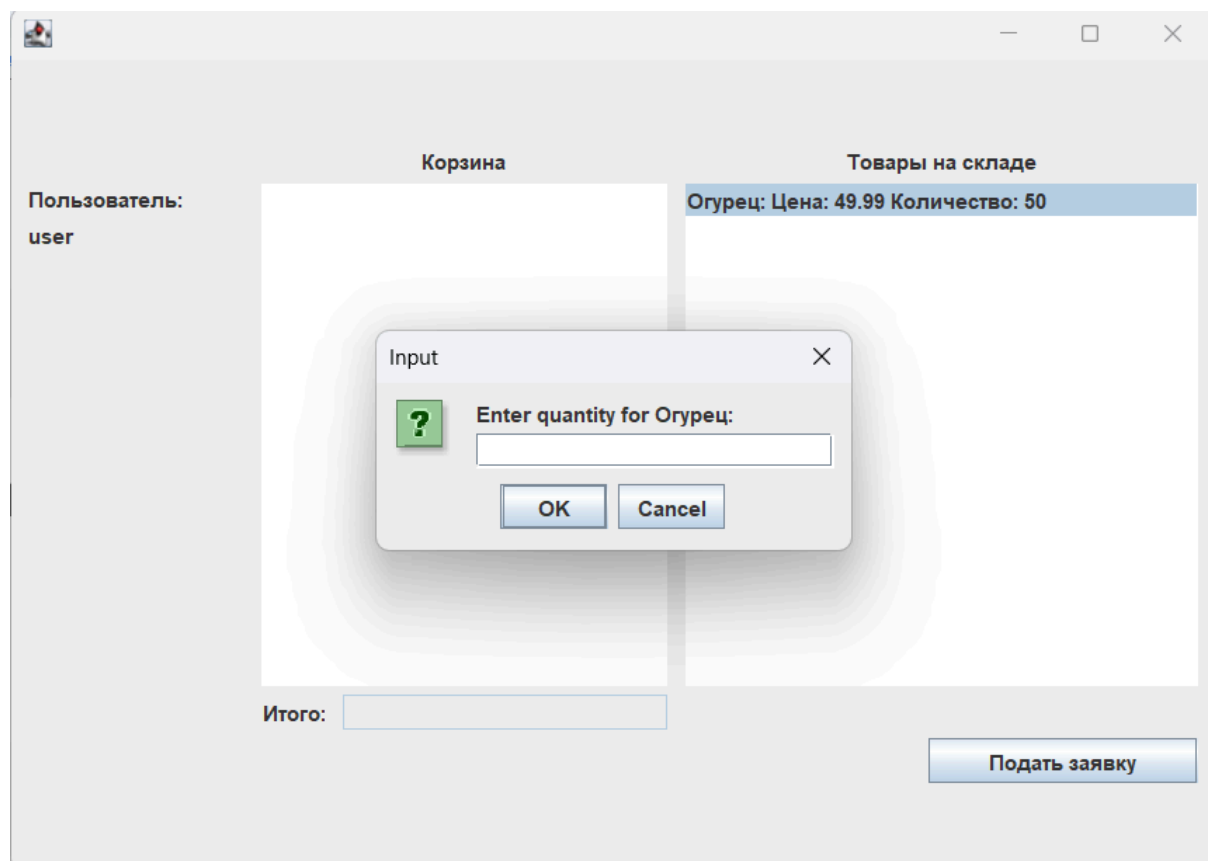


Рисунок 7– Окно с выбором количества заказываемого товаров

The screenshot shows a web application window with a light gray background. On the left, there is a label "Пользователь:" followed by the text "user". In the center, there is a section titled "Корзина" (Cart) containing a table with one row: "Огурец: Цена: 49.99 Количество: 3". Below this table, there is a label "Итого:" followed by a text box containing the value "149.97". On the right, there is a section titled "Товары на складе" (Goods in stock) containing a table with one row: "Огурец: Цена: 49.99 Количество: 47". At the bottom right, there is a blue button with the text "Подать заявку" (Submit application).

| Корзина |                           |
|---------|---------------------------|
| Огурец: | Цена: 49.99 Количество: 3 |

Итого: 149.97

| Товары на складе |                            |
|------------------|----------------------------|
| Огурец:          | Цена: 49.99 Количество: 47 |

Подать заявку

Рисунок 8 - Добавление товара в корзину и подсчет итоговой стоимости

После заполнения всех пунктов, необходимо нажать кнопку “Подать заявку”. После успешного оформления заявки (Рисунок 8). Сформированный заказ будет добавлен в базу данных с пометкой статуса “Новая”.

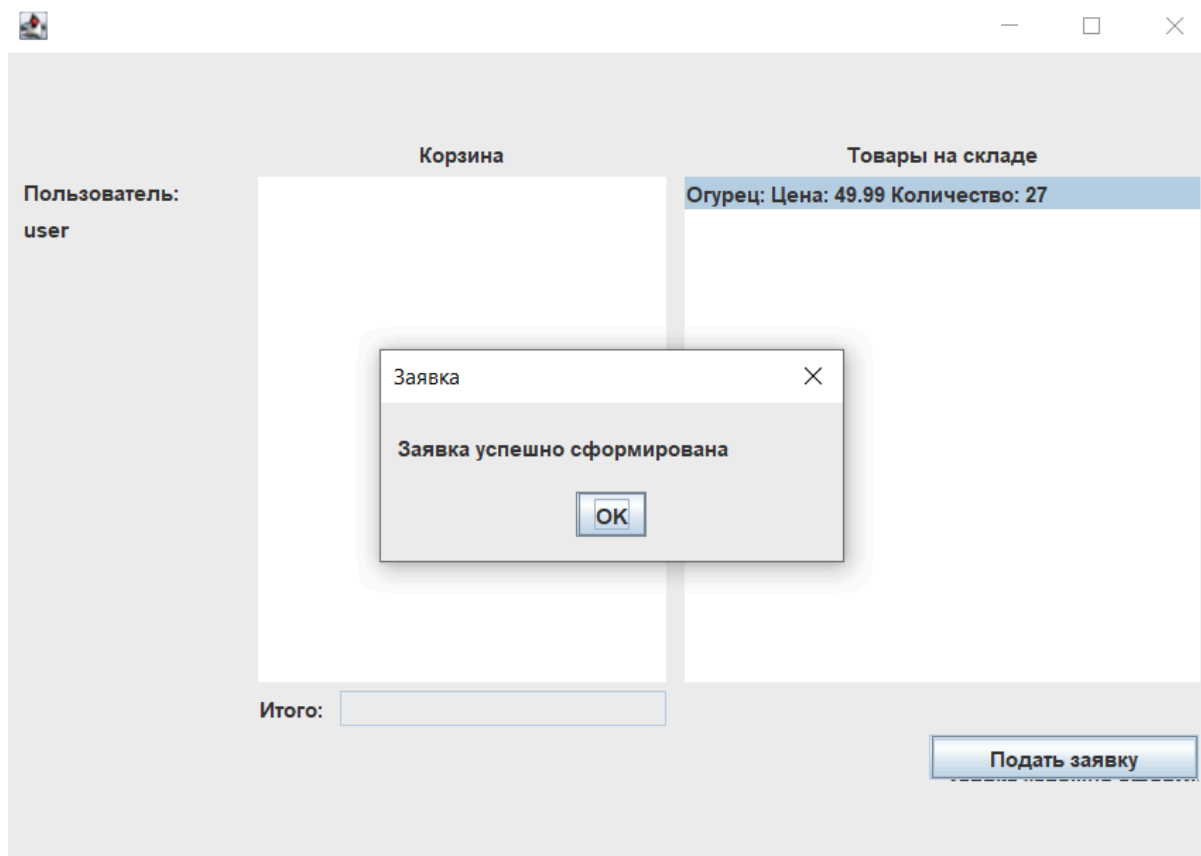


Рисунок 9 – успешное оформление заказа.

Если работник торговой фирмы успешно авторизовался, то на экране отобразится окно (Рисунок 10).

В список «Заказы» из базы данных появятся оформленные клиентами заказы с любым статусом. Если заказ в состоянии «Новая», то при двойном нажатии по такому заказу откроется окно с выбором взаимодействия с заказом (Рисунок 11). Это окно будет содержать дополнительные две кнопки «Отпустить товар» и «Не отпускать». При нажатии на кнопку «Отпустить товар» заказ перейдет в состояние «Завершена» и появится соответствующее уведомление (Рисунок 12), если данные были успешно занесены в базу данных. Если же не отпускать товар, то состояние заказа будет «Отменено» и появится также соответствующее уведомление о том, что отпуск товара был отменён. Если же товар находится в других состояниях, то будет аналогичное окно с информацией и кнопкой «Ок».

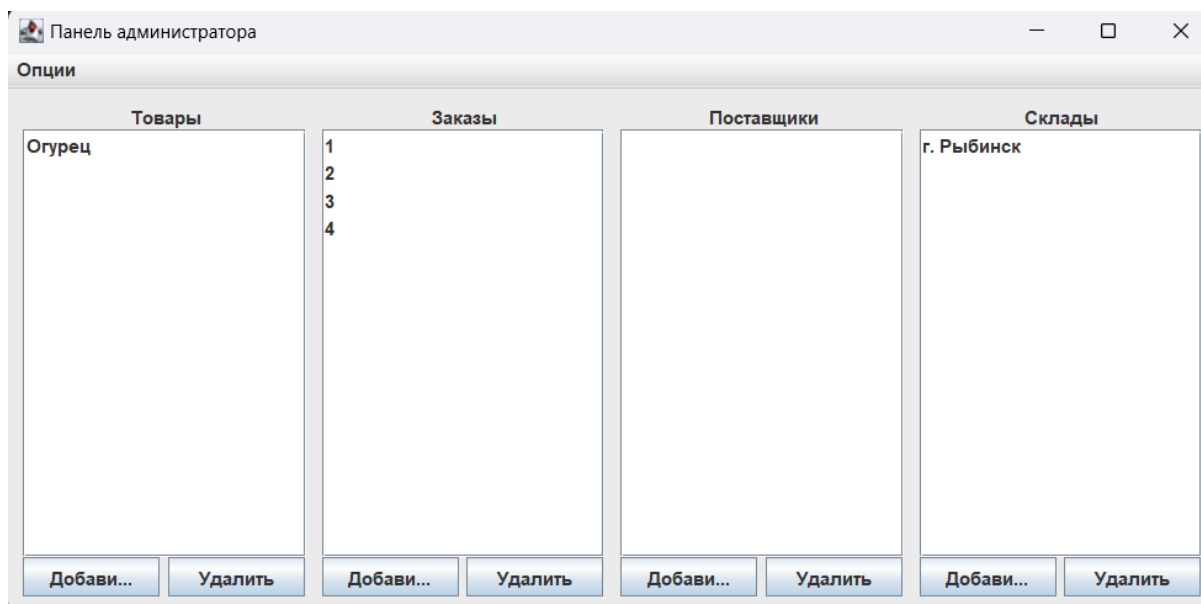


Рисунок 10 – Окно работника торговой фирмы.

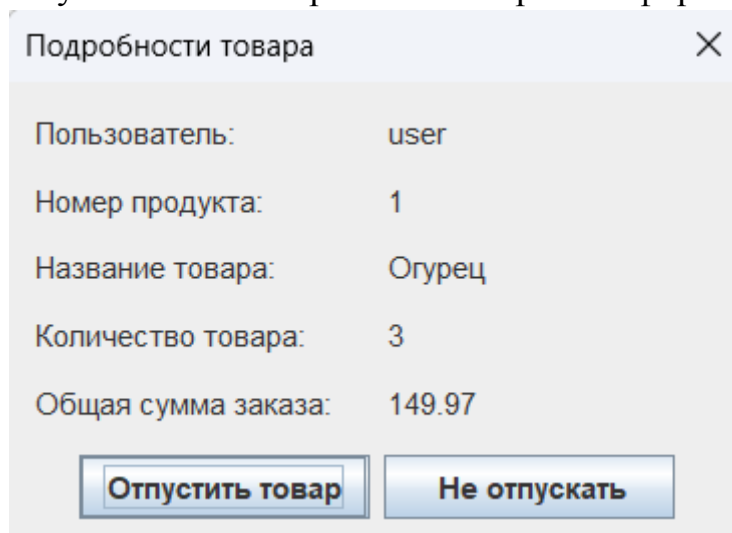


Рисунок 11 - окно с отпуском товара

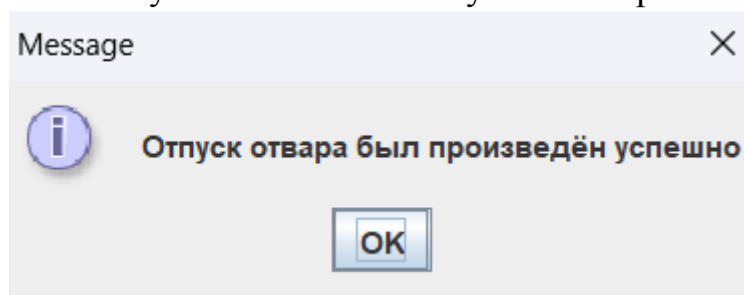


Рисунок 12 - пример отпуска товара

У работников торговой фирмы есть определённый набор функций для удобного взаимодействия с данными товаров, поставщиков, складов и заказов.

У каждого списка есть дополнительные кнопки “Добавить” и “Удалить”:

- Добавить товар;
- Добавить заказ;
- Добавить склад
- Добавить поставщика
- Удалить товар
- Удалить склад
- Удалить заказ
- Удалить поставщика

### **Добавить товар.**

При нажатии на кнопку «Добавить» под списком “Товары” откроется соответствующее окно формирования нового товара (Рисунок 13).

Данные поля “Название” будут проверены на уникальность и если товар будет назван уникальным именем, а также все поля будут заполнены, то данные будут записаны в базу данных.

Если данные были введены верно, то появится соответствующее окно с тем, что данные успешно записаны в базу данных (Рисунок 14)

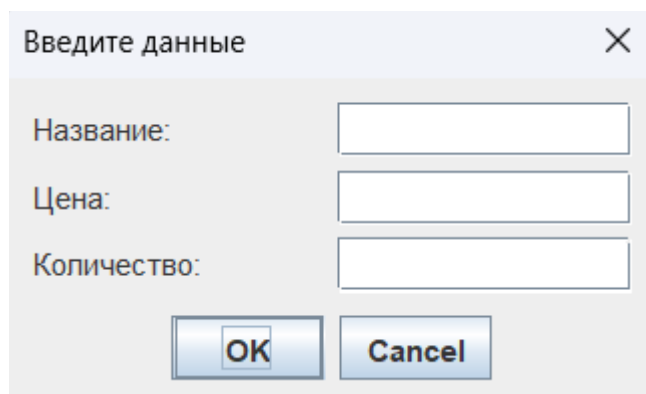


Рисунок 13–Добавить товар

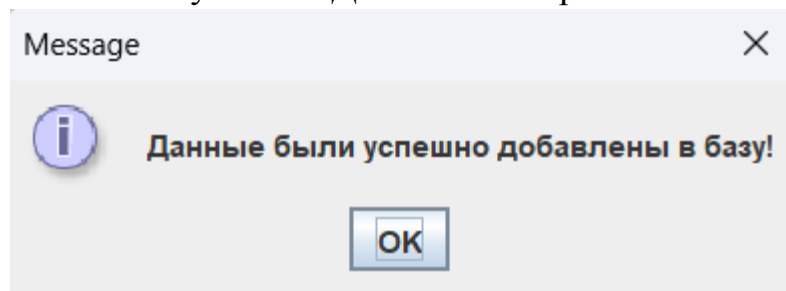


Рисунок 14 - успешное добавление товара в базу данных

## Удалить товар.

При нажатии на кнопку «Удалить» под списком «Товары» при корректном удалении товара из базы данных отобразится сообщение (Рисунок 15).

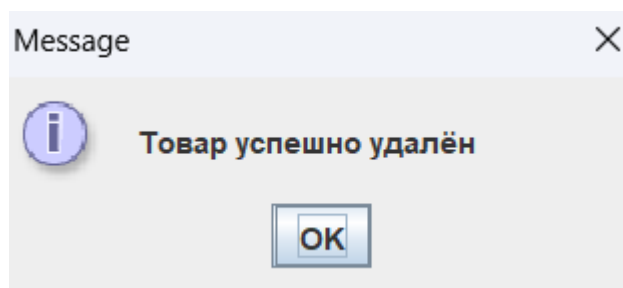


Рисунок 15 - удаление товара

**Примечание:** Остальные пункты меню работают аналогичным образом, описанным ранее

## Опции.

При нажатии на кнопку выпадающего, находящуюся над списками, отобразятся два пункта меню: «Получить отчёт» и «Отобразить график».

## Получить отчёт

При выборе пункта подменю «Получить отчет», если при этом, пользователь выбрал какой-либо пункт меню в одном из списков, то сформируется отчет в формате .csv, представляющий собой excel формат

|    | C1                | C2                                                    | C3                | C4                   | C5             | C6      |
|----|-------------------|-------------------------------------------------------|-------------------|----------------------|----------------|---------|
| 1  | Покупатель:       | user                                                  | <unset>           | <unset>              | <unset>        | <unset> |
| 2  | Адрес:            | Ягутка                                                | <unset>           | <unset>              | <unset>        | <unset> |
| 3  | Поставщик:        | Общество с ограниченной ответственностью «Мир цвет... | <unset>           | <unset>              | <unset>        | <unset> |
| 4  | Адрес:            | Волгоградская область                                 | с. Ивановка       | ул. Зеленая          | д. 1           | <unset> |
| 5  | Дата:             | 2024-05-21 10:22:56.777                               | <unset>           | <unset>              | <unset>        | <unset> |
| 6  | Статус заказа:    | COMPLETED                                             | <unset>           | <unset>              | <unset>        | <unset> |
| 7  | Срок поставки:    | 2024-05-21 10:22:56.777                               | <unset>           | <unset>              | <unset>        | <unset> |
| 8  | <null>            | <unset>                                               | <unset>           | <unset>              | <unset>        | <unset> |
| 9  | N п/п             | Наименование товара                                   | Количество товара | Цена за единицу руб. | Стоимость руб. | <null>  |
| 10 | 1                 | Огурец                                                | 4                 | 49.99                | 199.96         | <null>  |
| 11 | <null>            | <unset>                                               | <unset>           | <unset>              | <unset>        | <unset> |
| 12 | Итого:            | <null>                                                | 4                 | <null>               | 199.96         | <null>  |
| 13 | <null>            | <unset>                                               | <unset>           | <unset>              | <unset>        | <unset> |
| 14 | <null>            | <unset>                                               | <unset>           | <unset>              | <unset>        | <unset> |
| 15 | Условия поставки: | <unset>                                               | <unset>           | <unset>              | <unset>        | <unset> |
| 16 | Примечания:       | <unset>                                               | <unset>           | <unset>              | <unset>        | <unset> |
| 17 | <null>            | <unset>                                               | <unset>           | <unset>              | <unset>        | <unset> |

## Отобразить график

При выборе пункта подменю “Отобразить график”, работнику торговой фирму будет отображен отчет в виде круговой диаграммы в отдельном окне, которые будет отображать процентное соотношение товаров, которые доступны фирме(Рисунок 16).

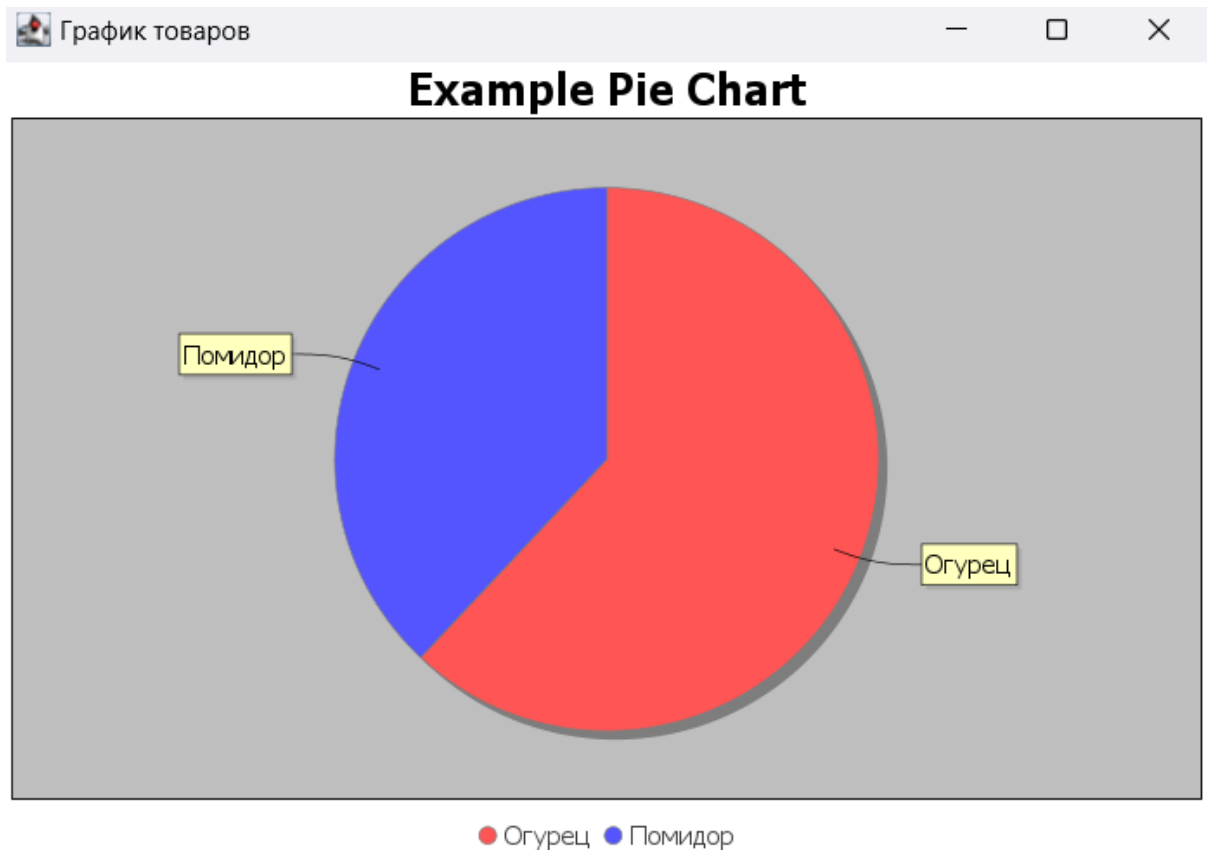


Рисунок 16 – Круговая диаграмма «Количество товара».



## 8. Заключение

В результате проделанной работы была спроектирована реляционная база данных на основе построения ER–диаграммы сущность-связь, составлены предварительные отношения по спроектированной диаграмме.

В процессе создания программы были получены навыки и новые знания в работе с кроссплатформенным фреймворком Swing, работающем с языком высокого уровня Java. Улучшены навыки работы с декларативным языком программирования SQL, а также был освоен движок для кроссплатформенных баз данных PostgreSQL.

В процессе работы было создано приложение для торговой фирмы, обеспечивающее автоматизацию некоторых действий.

## 9. Список литературы

1. Иван Портянкин Java Swing: Эффективные пользовательские интерфейсы. - 2-е изд. - Москва, Шмитовский пер: Лори, 2011. - 591 с.
2. Домбровская Генриэтта, Бейликова Анна, Новиков Борис  
Оптимизация запросов PostgreSQL. - 1-е изд. - Москва, Шмитовский пер: ДМК-Пресс, 2022. - 278 с.

## 10. Приложения

### Приложение 1:

Приложение N 1  
к Договору поставки  
от 20.01.2017 N 32

#### ЗАЯВКА на поставку товара

25 февраля 2017 года

Покупатель: Общество с ограниченной ответственностью «Ромашка»  
Адрес: г. Волгоград, ул. Ленина, д. 1

Поставщик: Общество с ограниченной ответственностью «Мир цветов»  
Адрес: Волгоградская область, с. Ивановка, ул. Зеленая, д. 1

На основании п. 2.1 Договора поставки от 20.01.2017 N 32 Покупатель просит осуществить поставку партии товара в следующем количестве и ассортименте:

| N<br>п/п | Наименование<br>товара    | Единица<br>измерения  | Количество<br>товара | Цена за<br>единицу,<br>руб. | Стоимость,<br>руб. |
|----------|---------------------------|-----------------------|----------------------|-----------------------------|--------------------|
| 1        | Тюльпаны микс<br>(Россия) | Коробка<br>(500 штук) | 1                    | 10000                       | 10000              |
| 2        | Розы микс<br>(Россия)     | Коробка<br>(600 штук) | 2                    | 20000                       | 40000              |
| ИТОГО:   |                           |                       | 3                    |                             | 50000              |

Срок поставки: до 04.03.2017.

Условия поставки: \_\_\_\_\_.

Примечания: \_\_\_\_\_.

Генеральный директор    *Королев*    / Королев В.С. /  
ООО «Ромашка»

## Приложение 2:

### Код создания диаграммы товаров

```
1 usage  👤 goldduck11
public PieGraph(List<Product> data) {
    super( title: "График товаров");
    this.data = data;
    setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE); // Устанавливаем DISPOSE_ON_CLOSE

    DefaultPieDataset dataset = createDataset();
    // Создаем диаграмму
    JFreeChart chart = createChart(dataset);
    // Создаем панель диаграммы и добавляем её в окно приложения
    ChartPanel chartPanel = new ChartPanel(chart);
    chartPanel.setPreferredSize(new Dimension( width: 560, height: 370));
    setContentPane(chartPanel);

    // Центрируем окно на экране
    setLocationRelativeTo(null);
}

1 usage  👤 goldduck11
private JFreeChart createChart(DefaultPieDataset dataset) {
    JFreeChart chart = ChartFactory.createPieChart(
        title: "Example Pie Chart", // заголовок диаграммы
        dataset,                    // данные
        true,                       // легенда
        true,                       // легенда
        urls: false);

    PiePlot plot = (PiePlot) chart.getPlot();
    // Настройка диаграммы (если требуется)

    return chart;
}
```

### Приложение 3:

Код выборки товаров и их количество на складе

```
1 usage  👤 Vladislav
public Map<Product, ProductInfo> getQuantityProduct() {
    try (Session session = sessionFactory.openSession()) {
        session.beginTransaction();
        String hql = "SELECT p, SUM(sp.quantity), p.price " +
            "FROM StorageProduct sp " +
            "JOIN Product p ON sp.productId = p.id " +
            "GROUP BY p.id, p.name, p.price";
        TypedQuery<Object[]> query = session.createQuery(hql, Object[].class);
        List<Object[]> results = query.getResultList();

        Map<Product, ProductInfo> productQuantities = new HashMap<>();

        for (Object[] result : results) {
            Product product = (Product) result[0];
            Long totalQuantity = (Long) result[1];
            Double price = (Double) result[2];

            productQuantities.put(product, new ProductInfo(totalQuantity, price));
        }

        for (Map.Entry<Product, ProductInfo> entry : productQuantities.entrySet()) {
            System.out.println("Product: " + entry.getKey().getName() +
                ", Total Quantity: " + entry.getValue().getQuantity() +
                ", Price: " + entry.getValue().getPrice());
        }
        session.getTransaction().commit();
        return productQuantities;
    }
}
```

## Приложение 4:

### Код создания заявки на приобретение товара

```
public void createRequisitionFromCart(List<ProductCert> productCerts, User user, double totalPrice) {
    try (Session session = sessionFactory.openSession()) {
        session.beginTransaction();
        Requisition requisition = new Requisition();
        requisition.setDate(new Date());
        requisition.setStatus(RequisitionStatus.NEW.getString());
        requisition.setUser(user);

        session.save(requisition);

        Long requisitionId = requisition.getId();

        for (ProductCert productCert : productCerts) {
            // Проверка текущего количества товара на складе
            String hql = String.format("SELECT sp FROM StorageProduct sp WHERE sp.productId = %s", productCert.getProduct().getId());
            TypedQuery<Object[]> query = session.createQuery(hql, Object[].class);
            List<Object[]> results = query.getResultList();

            if (results == null) {
                session.getTransaction().rollback();
                throw new IllegalArgumentException("Product not found in storage: " + productCert.getProduct().getName());
            }

            StorageProduct storageProduct = ((StorageProduct) results.get(0)[0]);
            Long currentQuantity = storageProduct.getQuantity();
            Long requestedQuantity = productCert.getProductInfo().getQuantity();

            if (requestedQuantity > currentQuantity) {
                session.getTransaction().rollback();
                throw new IllegalArgumentException("Not enough stock for product: " + productCert.getProduct().getName());
            }

            // Вычитание количества товара из базы данных
            Long newQuantity = currentQuantity - requestedQuantity;
            storageProduct.setQuantity(newQuantity);
            session.update(storageProduct);

            RequisitionProduct requisitionProduct = new RequisitionProduct();
            requisitionProduct.setRequisitionId(requisitionId);
            requisitionProduct.setProductId(productCert.getProduct().getId());
            requisitionProduct.setQuantity(productCert.getProductInfo().getQuantity());
            requisitionProduct.setPrice(totalPrice);

            session.save(requisitionProduct); // Сохранение товара заявки в базе данных
        }

        session.getTransaction().commit();
    }
}
```

## Приложение 5:

### Код отмены заявки от пользователя

```
public void cancelRequisition(Long requisitionId) {
    try (Session session = sessionFactory.openSession()) {
        session.beginTransaction();

        // Получение заявки
        Requisition requisition = session.get(Requisition.class, requisitionId);
        if (requisition == null) {
            session.getTransaction().rollback();
            throw new IllegalArgumentException("Requisition not found: " + requisitionId);
        }

        // Получение списка товаров в заявке
        String hql = "FROM RequisitionProduct rp WHERE rp.requisitionId = :requisitionId";
        List<RequisitionProduct> requisitionProducts = session.createQuery(hql, RequisitionProduct.class)
            .setParameter(s: "requisitionId", requisitionId)
            .getResultList();

        // Возврат количества товаров на склад
        for (RequisitionProduct requisitionProduct : requisitionProducts) {
            String productHql = "FROM StorageProduct sp WHERE sp.productId = :productId";
            StorageProduct storageProduct = session.createQuery(productHql, StorageProduct.class)
                .setParameter(s: "productId", requisitionProduct.getProductId())
                .uniqueResult();

            if (storageProduct == null) {
                session.getTransaction().rollback();
                throw new IllegalArgumentException("Product not found in storage: " + requisitionProduct.getProductId());
            }

            long newQuantity = storageProduct.getQuantity() + requisitionProduct.getQuantity();
            storageProduct.setQuantity(newQuantity);
            session.update(storageProduct);
        }

        session.getTransaction().commit();
    } catch (Exception e) {
        e.printStackTrace();
        session.getTransaction().rollback();
    }
}
```